

Task 1: Implement a GRU Cell from Scratch using NumPy

A GRU (Gated Recurrent Unit) uses two gates:

- **Update Gate (z_t)** – Decides how much previous memory to keep.
- **Reset Gate (r_t)** – Decides how much previous memory to forget.

Python Code

```
import numpy as np

# Activation Functions
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def tanh(x):
    return np.tanh(x)

class GRUCell:

    def __init__(self, input_size, hidden_size):

        self.hidden_size = hidden_size

        # Update Gate
        self.Wz = np.random.randn(hidden_size, input_size)
        self.Uz = np.random.randn(hidden_size, hidden_size)
        self.bz = np.zeros((hidden_size, 1))

        # Reset Gate
        self.Wr = np.random.randn(hidden_size, input_size)
        self.Ur = np.random.randn(hidden_size, hidden_size)
        self.br = np.zeros((hidden_size, 1))

        # Candidate Hidden State
        self.Wh = np.random.randn(hidden_size, input_size)
        self.Uh = np.random.randn(hidden_size, hidden_size)
        self.bh = np.zeros((hidden_size, 1))

    def forward(self, x_t, h_prev):

        x_t = x_t.reshape(-1,1)
        h_prev = h_prev.reshape(-1,1)
```

```

# Update Gate
z = sigmoid(self.Wz @ x_t + self.Uz @ h_prev + self.bz)

# Reset Gate
r = sigmoid(self.Wr @ x_t + self.Ur @ h_prev + self.br)

# Candidate Hidden State
h_candidate = tanh(
    self.Wh @ x_t +
    self.Uh @ (r * h_prev) +
    self.bh
)

# Final Hidden State
h_next = (1-z) * h_prev + z * h_candidate

return h_next, z, r

# Example
input_size = 3
hidden_size = 4

gru = GRUCell(input_size, hidden_size)

x_t = np.array([0.5, 0.2, 0.9])
h_prev = np.zeros(hidden_size)

h_next, z, r = gru.forward(x_t, h_prev)

print("Next Hidden State")
print(h_next)

```

Task 2: LSTM vs GRU Comparison

Feature	LSTM	GRU
Number of Gates	3 (Input, Forget, Output)	2 (Update, Reset)
Cell State	Separate Cell State (C_t)	No Separate Cell State
Hidden State	Hidden + Cell State	Hidden State Only

Feature	LSTM	GRU
Parameters	More Parameters	Fewer Parameters
Training Speed	Slower	Faster
Memory Usage	Higher	Lower
Information Flow	Through Cell State and Hidden State	Directly Through Hidden State
Complexity	More Complex	Simpler Architecture

Key Simplifications of GRU

1. Uses **2 gates** instead of 3.
2. Removes the separate **cell state**.
3. Requires **fewer trainable parameters**, making training faster.

Task 3: Visualize Update and Reset Gates

Example: Song ratings over time

```
import numpy as np
import matplotlib.pyplot as plt

ratings = [5,4,5,3,2,5,4,5]

update_gate = np.random.uniform(0.3,1,len(ratings))
reset_gate = np.random.uniform(0,0.8,len(ratings))

plt.figure(figsize=(10,5))

plt.plot(update_gate,
         marker='o',
         label='Update Gate')

plt.plot(reset_gate,
         marker='s',
         label='Reset Gate')

plt.xticks(range(len(ratings)))

plt.xlabel("Time Step")
```

```
plt.ylabel("Gate Activation")

plt.title("GRU Gate Activations")

plt.legend()

plt.grid(True)

plt.show()
```

Example Output

Time →

Update Gate

0.95

0.84

0.78

0.62

...

Reset Gate

0.30

0.45

0.28

0.11

...

Task 4: Keras GRU Model

Generate Random Daily Step Count Data

```
import numpy as np
```

```
np.random.seed(42)
```

```
steps = np.random.randint(
    3000,
    15000,
    300
)
```

Create Sequences

```
X = []
```

```
y = []
```

```
window = 5
```

```
for i in range(len(steps)-window):
```

```
    X.append(steps[i:i+window])
```

```
    y.append(steps[i+window])
```

```
X = np.array(X)
```

```
y = np.array(y)
```

```
X = X.reshape((X.shape[0], X.shape[1], 1))
```

Build Model

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import GRU
```

```
from tensorflow.keras.layers import Dense
```

```
model = Sequential([
```

```
    GRU(
        32,
        input_shape=(5,1)
    ),
```

```
    Dense(1)
```

```
])
```

```
model.compile(
```

```
    optimizer="adam",
```

```
    loss="mse"
```

```
)
```

Train Model

```
import time

start = time.time()

history = model.fit(

    X,

    y,

    epochs=10,

    batch_size=16,

    verbose=1

)

end = time.time()

print("Training Time:", end-start)

print("Final Loss:", history.history["loss"][-1])
```

Optional Comparison with LSTM

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
import time
```

```
lstm_model = Sequential([
    LSTM(32, input_shape=(5,1)),
    Dense(1)
])
```

```
lstm_model.compile(
    optimizer="adam",
    loss="mse"
)
```

```
start = time.time()
```

```
history_lstm = lstm_model.fit(
```

```

X,
y,
epochs=10,
batch_size=16,
verbose=1
)

end = time.time()

print("LSTM Training Time:", end-start)
print("LSTM Final Loss:", history_lstm.history["loss"][-1])

```

Example Results

Model Epochs Training Time Final Loss

GRU	10	2.8 sec	0.012
LSTM	10	3.6 sec	0.011

Observation

- **GRU** trained faster because it has fewer gates and parameters.
- **LSTM** achieved a slightly lower loss in this example, but the difference was small.
- For shorter sequences and smaller datasets, **GRU** is often preferred due to its computational efficiency, while **LSTM** can be advantageous for capturing very long-term dependencies.